

DSAC-Configured Differential Evolution for Cloud–Edge–Device Collaborative Task Scheduling

Yuanjun Laili , Member, IEEE, Xiaohan Wang , Lin Zhang , Senior Member, IEEE, and Lei Ren , Member, IEEE

Abstract—Industrial Internet of Things enables various manufacturing processes executed in distributed production lines and flexible workshops. With different cloud–edge–device collaboration ways, interconnected manufacturing tasks and computational tasks are cooperatively completed in manufacturing cells, cloud resources, and edge resources. Large-scale decision variables and complex precedence constraints make the scheduling problem intractable. To this end, this article proposed a discretized soft actor–critic configured differential evolution algorithm to find a stable solution for the cloud–edge–device collaborative task-scheduling problem. A mathematical model is established to describe the relationship between different tasks, the variables, the main constraints in collaboration, and the scheduling targets. A decentralized partially observable Markov decision process is modeled with five neural networks and three discretized loss functions to formulate the discretized soft actor–critic policy efficiently and enable it to find the best differential evolution configurations for different scheduling cases. Experimental analysis of four cloud–edge–device scheduling instances indicates that the proposed method trained in one case is adaptable to the other three cases. In the four cases, the proposed method reduces the total objective by 30.82% and 44.35% at most compared to five deep-reinforcement-learning-based differential evolution algorithms and seven typical evolutionary algorithms, respectively.

Index Terms—Cloud–edge–device collaboration, deep reinforcement learning (DRL), differential evolution (DE), Industrial Internet of Things (IIoT), task scheduling.

Manuscript received 3 April 2023; accepted 23 May 2023. Date of publication 31 May 2023; date of current version 19 January 2024. This work was supported by the National Key Research and Development Program of China under Grant 2021YFB3300400 and in part by the National Natural Science Foundation of China under Grant 62173017 and Grant 62225302. Paper no. TII-23-1160. Paper no. TII-23-1160. (Corresponding authors: Xiaohan Wang; Lei Ren.)

Yuanjun Laili and Lei Ren are with the School of Automation Science and Electrical Engineering, Beihang University, Beijing 100191, China, and also with the Zhongguancun Laboratory, Beijing 100094, China (e-mail: lailiyuanjun@buaa.edu.cn; renlei@buaa.edu.cn).

Xiaohan Wang and Lin Zhang are with the School of Automation Science and Electrical Engineering, Beihang University, Beijing 100191, China (e-mail: by1903042@buaa.edu.cn; zhanglin@buaa.edu.cn).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TII.2023.3281661>.

Digital Object Identifier 10.1109/TII.2023.3281661

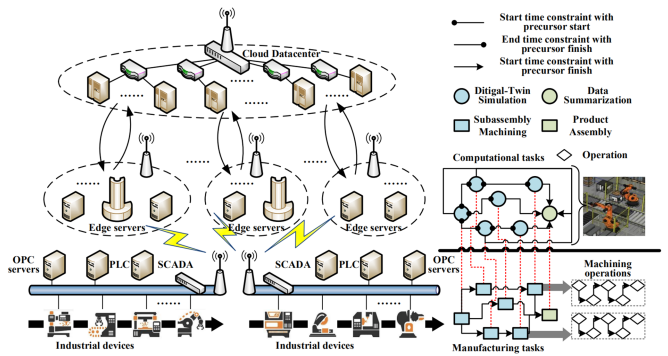


Fig. 1. Structure of IIoT and a manufacturing example with the digital twin.

I. INTRODUCTION

INDUSTRIAL Internet of Things (IIoT) closely connects manufacturing resources, suppliers, products, and customers with cloud and edge [1]. A general structure of IIoT is shown in Fig. 1. Cloud data center provides abundant computing resources and storage for industrial data management, multidisciplinary knowledge extraction, and complex computation. Edge resources nearby industrial fields alleviate the pressure of the cloud and make real-time supervision and control under the assistance of the cloud [2].

There are two kinds of tasks to be scheduled in an IIoT-based manufacturing system: 1) manufacturing tasks and 2) computational tasks. A manufacturing task defines the machining or the assembly process, which can be finished by a production line or a workshop. In a general production scenario, each manufacturing task is further divided into a series of operations. These operations have strict precedence relationships and can be executed distributively by different manufacturing devices.

To implement efficient production, each manufacturing task is usually accompanied by a group of computational applications, such as machining control, data sampling, numerical analysis, or digital twin. In traditional manufacturing systems, these computation applications are deployed on OPC (OLE for Process Control) servers (servers in the OLE for Process Control-Unified Architecture (OPC-UA) architecture [3]) or Programmable Logic Controller

(PLC) servers locally. In an IIoT platform, they can be combined as a computational task offloading to an edge server or a cloud server. Therefore, a manufacturing task is usually associated with a computational task.

Consider tasks in Fig. 1 as an example, there are six machining tasks (blue rectangular) producing three subassemblies and one assembly task (green rectangular) producing a product. Each machining task corresponds to a digital twin simulation (blue circle) while the last assembly task is associated with a data summarization task to predict product quality. When the first digital twin simulation task is initiated, the rest five digital twin simulation tasks should be started to collect environmental data and establish simulation as well. The six digital twin tasks should be carried out cooperatively and finished simultaneously to demonstrate a complete production process of the three subassemblies. Then, the data summarization task collects data from the six digital twin simulation tasks and starts assembly analysis. Obviously, the six digital twin simulation tasks have a start-time constraint with a precursor start and an end-time constraint with a precursor finish while the data summarization task has a start time constraint with precursor finish. The connection relationships between such computational tasks are complex. The communication overhead among cloud, edge, and manufacturing resources increases largely [4], [5].

In addition, a computational task requires data from one or more manufacturing devices and data synchronization with these devices. If a cloud server executes the task, its input and output are collected and deployed by an edge server. On the contrary, if an edge server executes the task, it applies empirical knowledge and historical data from the cloud data center to implement fast computing.

With different cloud–edge collaboration modes, numerous tasks interconnected in complex ways are submitted to the IIoT platform sporadically. These task requests come from complex product suppliers for producing components and subassemblies. As a single complex product (such as an electromobile or automobile) has hundreds or even thousands of components and subassemblies, an IIoT platform that connects hundreds of industrial devices and edge servers can accept at least one hundred manufacturing tasks at a time. Suppose each task includes five basic operations, there are at least 500 variables to determine the devices for finishing these manufacturing tasks, 500 variables to determine the priority of their operations, 100 variables to decide whether the corresponding computational tasks are offloaded to edge or cloud, and 100 variables to find the suitable server for executing the computational tasks. Thousands of variables make the scheduling of hybrid tasks in the IIoT platform more challenging [6].

Existing research on task scheduling in the IIoT environment focuses on either the computational task-scheduling problems in the cloud–edge environment, or the manufacturing task-scheduling problems in the industrial field [7]. Little research considered the complex interconnections between these two sides. The tasks can form a cyclic graph with complex start-time and end-time constraints. Hence, finding problem-specific heuristics and methods for scheduling scenarios with different complex precedence constraints is nontrivial.

Therefore, this article establishes a mathematical model of the hybrid task-scheduling problem in the IIoT environment considering three task precedence constraints. A deep-reinforcement-learning (DRL) configured differential evolution (DE) algorithm is designed to solve this problem. The evolutionary process is modeled as a decentralized partially observable Markov decision process (Dec-POMDP). Then, a discretized soft actor–critic (DSAC) policy is established for selecting DE mutation operators. In contrast to other DRL algorithms integrated into the DE algorithm, the DSAC alleviates the overfitting issue and reduces the instability in the DE algorithm via maximum entropy learning. Five neural networks and three loss functions are designed to train the DSAC policy efficiently. Experimental analysis of four cases shows that the proposed DSAC-DE trained in one case is always adaptable to the other three cases. In the four cases, the proposed DSAC-DE reduces the overall manufacturing time and energy consumption by 44.35% and 30.82% at most compared to seven typical EAs and five DRL-integrated DE-based (DRL-DE) algorithms, respectively.

The rest of this article is organized as follows: Section II presents the recent works on the research of task scheduling in the IIoT environment. Section III gives the mathematical model of cloud–edge–device collaborative task scheduling in IIoT. Section IV introduces the details of the proposed DSAC-DE algorithm. Experimental results are discussed in Section V. Finally, Section VI concludes this article.

II. RELATED WORKS

This section gives a brief overview of the research on task scheduling in the IIoT environment from two aspects: 1) problem modeling and 2) method design.

A. Task-Scheduling Models in Cloud–Edge Collaboration

Computational task scheduling in fog, edge, and cloud computing has been widely studied [8], [9]. The characteristics of the hierarchical resource connection ways [6], different wireless transmission costs, and the network function virtualization rules that influence the computation and communication efficiency are introduced. Considering the collaboration between cloud and edge, Xie et al. [10] and Liu et al. [11] scheduled the computational tasks considering the communication of cloud, edge, and manufacturing devices. Nevertheless, the influence between manufacturing tasks and computational tasks is ignored.

In recent years, manufacturing task scheduling in flexible and distributed workshops has attracted considerable interest [12], [13]. The main target is to realize just-in-time manufacturing with due-time constraints [14]. The issues about production line reconfiguration, dynamic resource occupancy, and logistics among distributed workshops that influence the manufacturing efficiency are taken into consideration [15], [16], [17]. Various task precedence constraints under different production processes, including sequence, parallel, and loop, are analyzed [17]. However, the collaboration of cloud and edge for manufacturing tasks is not considered. The influence between manufacturing tasks and the related computational tasks is neglected.

B. Task-Scheduling Algorithms in the IIoT Environment

Commonly applied scheduling algorithms include deterministic mathematical programming methods, EAs, and DRL. Due to the high complexity, deterministic methods are usually applied as heuristics for small-scale local search [6]. EAs with population-based iteration are more adaptable for solving different task-scheduling problems [13], [15]. Yet the extensibility of a fixed EA for different scheduling scenarios is limited. To solve dynamic scheduling problems, DRL is introduced to learn policies under dynamic contexts and generate scheduling solutions [12]. However, DRL requires a long training process to obtain similar accuracy compared with classical EAs [18].

Recent research also applies DRL as a hyperheuristic to adjust EAs in solving task-scheduling problems [19]. Typical studies include the use of deep Q-learning network (DQN) and actor-critic (AC) models to adjust hyperparameters of GAs, variable neighborhood search, particle swarm optimization (PSO), DEs, and so forth [20], [21], [22]. Based on these studies, Zhang et al. [23] adopted a radial basis network as a surrogate to expand the dataset of DQN. Sharma et al. [24] defined complex state representation with 99 variables and three different reward functions to train the DQN. Xu et al. [25] defined the reward as the improvement of the current state and the next state during the evolutionary search. However, all of these works model the evolutionary process as a Markov decision process (MDP) without random noise. However, the randomness in the evolutionary search process always makes the training of DRL nontrivial, but none of these works discussed this issue.

III. CLOUD-EDGE-DEVICE COLLABORATIVE TASK-SCHEDULING MODEL IN THE IIOT ENVIRONMENT

This section gives a mathematical representation of the cloud-edge-device collaborative task-scheduling model in the IIoT environment. The main notations used in this article are listed in Table I.

The cloud-edge-device collaborative task-scheduling problem means assigning manufacturing devices, edge servers, and cloud servers to execute the collaborative computational tasks and manufacturing tasks. Suppose there are N manufacturing tasks $\mathbf{J} = \{J_1, J_2, \dots, J_N\}$ and N computational tasks $\mathbf{C} = \{C_1, C_2, \dots, C_N\}$ submitted to the IIoT platform at a time.

A manufacturing task J_i contains n_i operations of $\mathbf{O}_i = \{O_{i,1}, O_{i,2}, \dots, O_{i,n_i}\}$. These operations are executed in sequence. Each operation is processed by a manufacturing device. There are a set of available manufacturing devices $\mathbf{D} = \{D_1, D_2, \dots\}$ in the IIoT platform. Each device $D_l \in \mathbf{D}$ has a task queue $Q_l^{(D)}$ by priority. The N manufacturing tasks can be executed independently.

A computational task C_i serves for a manufacturing task J_i can be described by six properties, i.e., $\{R_{\text{cpu},i}, R_{\text{mem},i}, t_i, H_i^{(\text{up})}, H_i^{(\text{down})}, \Lambda_i\}$, specifying the required amount of CPU cores and memory size for C_i , the basic computational time, the amount of data required from the n_i devices for executing J_i , the amount of knowledge required from the cloud data center, and the neighbor tasks that communicate with C_i , respectively.

TABLE I
PRIMARY NOTATIONS

Notation	Meaning
N	Number of manufacturing tasks
J_i	i th manufacturing task
n_i	Number of operations in J_i
$\mathbf{J}$	N manufacturing tasks
$\mathbf{C}$	N computational tasks
C_i	i th computational task
$O_{i,j}$	j th operations of J_i
$\mathbf{O}_i$	n_i operations in J_i
$\mathbf{D}$	Set of manufacturing devices
D_l	l th manufacturing device in $\mathbf{D}$
$\mathbf{D}_{i,j}$	Candidate device set for operating $O_{i,j}$
$\mathbf{E}_i$	Candidate edge server set for executing C_i
$\mathbf{S}$	Candidate cloud servers for computational tasks
$X_{i,j}$	Selected device for operating $O_{i,j}$
$Y_{i,j}$	Priority of $O_{i,j}$
B_i	Cloud/edge offloading flag for C_i
Z_i	Cloud/edge server for executing C_i
$Q_l^{(D)}$	Task queue of the device D_l
$Q_{X_{i,j}}^{(D)}$	Task queue of the device $X_{i,j}$
$T_{O_{i,j}}^{(\text{start})}$	Start time of $O_{i,j}$
$T_{O_{i,j}}^{(\text{end})}$	End time of $O_{i,j}$
$T_{J_i}^{(\text{end})}$	End time of J_i
$T_{X_{i,j-1}, X_{i,j}}^{(\text{trans})}$	Transportation time between $X_{i,j-1}$ and $X_{i,j}$
$T_{O_{i,j}}^{(\text{proc})}$	Processing time of $O_{i,j}$
$T^{(m)}$	Makespan of the N manufacturing tasks
$W^{(m)}$	Energy consumption of the N manufacturing tasks
$P_l^{(D)}$	Average power of the device D_l
$P_{Z_i}^{(c)}$	Average power of the server Z_i
t_i	Basic computational time of C_i
$R_{\text{cpu},i}$	Required CPU cores for C_i
$R_{\text{mem},i}$	Required memory size for C_i
$H_i^{(\text{up})}$	Amount of data required by n_i devices for executing J_i
$H_{i,l}^{(\text{up})}$	Amount of data required by the l -th device, $l \in [1, n_i]$
$H_i^{(\text{down})}$	Amount of data required by the cloud data center
$\Lambda_i^{(\kappa)}$	Precursors of C_i with the κ -th type precedence constraint
Λ_i	Neighbor tasks of C_i
$V_{\text{cpu},k}^{(\text{cloud/edge})}$	CPU cores of the k -th cloud/edge server
$V_{\text{mem},k}^{(\text{cloud/edge})}$	Memory size of the k -th cloud/edge server
$r_i(X_{i,j})$	Wireless transmission rate between $X_{i,j}$ and Z_i
w	Basic channel bandwidth of a 5G network
ϖ_0	Communication noise in a 5G network
$q_{X_{i,j}}$	Transmission power of device $X_{i,j}$
q_l	Transmission power of device D_l
$g_{X_{i,j},i}$	Channel gain from $X_{i,j}$ to the base station
$g_{l,i}$	Channel gain from D_l to the base station
$T_{\text{up},i}^{(\text{cloud/edge})}$	Data upload time for C_i
$T_{\text{down},i}^{(\text{edge})}$	Data download time for C_i
r_{cloud}	Bandwidth between an edge server and the cloud
$W_i^{(c)}$	Energy consumption of C_i
$T_{C_i}^{(\text{start})}$	Start time of C_i
$T_{C_i}^{(\text{end})}$	The end time of C_i
TC_i	The executing time of C_i

The length of $H_i^{(\text{up})}$ is n_i . The interconnection among N computational tasks in IIoT can be summarized into the following three types of execution constraints.

Type I precedence: A computational task can start only if the precursor tasks are finished.

Type II precedence: A computational task can start only if the precursor tasks are started.

Type III precedence: A computational task can finish only if the precursor tasks are finished.

Accordingly, the precursor tasks of the constraint $\kappa \in [1, 3]$ are represented by a set $\Lambda_i^{(\kappa)} \subset \Lambda_i$.

The candidate cloud servers for a computational task are represented by a set $\mathcal{S}$. The candidate edge servers for C_i are $\mathcal{E}_i$, which is a collection of available edge servers nearby $\mathcal{D}_{i,j}, \forall j \in [1, n_i]$. A server includes two main properties, i.e., $\{V_{\text{cpu},k}^{(\text{cloud/edge})}, V_{\text{mem},k}^{(\text{cloud/edge})}\}$, which represent the CPU cores and memory size of the server. A computational task C_i can be assigned to a server S_k or $E_{i,k}$ if $R_{\text{cpu},i} \leq V_{\text{cpu},k}^{(\text{cloud/edge})}$ and $R_{\text{mem},j} \leq V_{\text{mem},k}^{(\text{cloud/edge})}$ are satisfied.

Under this scenario, the decision variables are as follows.

- 1) $X_{i,j}, \forall i \in [1, N], j \in [1, n_i]$ determines the device for processing a manufacturing operation $O_{i,j}$. Each operation corresponds to a collection of candidate devices $\mathcal{D}_{i,j}$. It has $X_{i,j} \in \mathcal{D}_{i,j}$.
- 2) $Y_{i,j}, \forall i \in [1, N], j \in [1, n_i]$ indicates the priority of $O_{i,j}$. If multiple operations are assigned to a manufacturing device, the operations are executed from the highest priority to the lowest one.
- 3) $B_i, \forall i \in [1, N]$ determines whether a computational task C_i is processed on an edge server or a cloud server. $B_i = 1$ means C_i is assigned to a nearby edge while $B_i = 0$ means C_i is executed in cloud.
- 4) $Z_i, \forall i \in [1, N]$ indicates the server for executing C_i . if $B_i = 1$, $Z_i \in \mathcal{E}_i$, otherwise $Z_i \in \mathcal{S}$. The distance between any edge server in $\mathcal{E}_i$ and the device $X_{i,j}, \forall j \in [1, n_i]$ should be less than the maximum wireless transmission distance.

The following assumptions are made.

- 1) 1. Each manufacturing operation is processed by a device and cannot be divided further.
- 2) 2. Each computational task represents a virtual machine or a microservice with multiple computation applications for a manufacturing task.
- 3) 3. Each device executes one and only one operation at a time.
- 4) 4. Each cloud/edge server can execute multiple computational tasks if it is not overloaded, i.e., the CPU and memory capacities are not fully occupied.

A. Time and Energy Consumption by Manufacturing Tasks

An operation $O_{i,j}$ will be started if its precursor $O_{i,j-1}$ is finished and all of the operations in $\mathcal{Q}_{X_{i,j}}^{(D)}$ that have higher priority than $O_{i,j}$ are finished. Therefore, suppose the starting time of an operation $O_{i,j}$ is $T_{O_{i,j}}^{(\text{start})}$, it is calculated by

$$T_{O_{i,j}}^{(\text{start})} = \max \left(T_{O_{i,j-1}}^{(\text{end})} + T_{X_{i,j-1}, X_{i,j}}^{(\text{trans})}, \max_{l \in \mathcal{Q}_{X_{i,j}}^{(D)}, Y_l > Y_{i,j}} T_l^{(\text{end})} \right) \quad (1)$$

where $T_{X_{i,j-1}, X_{i,j}}^{(\text{trans})}$ is the transportation time between two devices that process the operations $O_{i,j-1}$ and $O_{i,j}$, respectively. Y_l is the priority of the l th operation in $\mathcal{Q}_{X_{i,j}}^{(D)}$.

Then, the end time of $O_{i,j}$ is calculated by

$$T_{O_{i,j}}^{(\text{end})} = T_{O_{i,j}}^{(\text{start})} + T_{O_{i,j}}^{(\text{proc})} \quad (2)$$

where $T_{O_{i,j}}^{(\text{proc})}$ is the basic processing time of $O_{i,j}$. The end time of a manufacturing task J_i equals the end time of its last operation, i.e., $T_{J_i}^{(\text{end})} = T_{O_{i,n_i}}^{(\text{end})}$. Therefore, the overall makespan of the N manufacturing tasks are calculated by

$$T^{(m)} = \max_{i \in [1, N]} T_{J_i}^{(\text{end})}. \quad (3)$$

Suppose $P_l^{(D)}$ is the average power of device D_l , the energy consumption of N manufacturing tasks is represented by

$$W^{(m)} = \sum_{l=1}^{|D|} P_l^{(D)} \cdot \left(T_{Q_{l,1}^{(D)}, Q_{l,|Q_l^{(D)}|}^{(D)}}^{(\text{end})} - T_{Q_{l,1}^{(D)}}^{(\text{start})} \right) \quad (4)$$

where $Q_{l,1}^{(D)}$ and $Q_{l,|Q_l^{(D)}|}^{(D)}$ are the first operation and the last operation in $\mathcal{Q}_l^{(D)}$, and $|Q_l^{(D)}|$ is the number of operations executed in D_l , $|D|$ is the number of devices in D .

B. Time and Energy Consumption by Computational Tasks

1) *Computational Tasks to Edge Servers*: If $B_i = 1$, the computational task C_i is executed by an edge server Z_i . The selected edge server Z_i for executing C_i has to receive data from n_i devices $\mathcal{X}_i = \{X_{i,1}, X_{i,2}, \dots, X_{i,n_i}\}$. According to the work in [26] and [27], the transmission rate $r_i(X_{i,j})$ between a device $X_{i,j}$ and an edge server Z_i is represented by

$$r_i(X_{i,j}) = w \log_2 \left(1 + \frac{q_{X_{i,j}} g_{X_{i,j},i}}{\varpi_0 + \sum_{l \in \mathcal{D}_i \setminus X_{i,j}} q_l g_{l,i}} \right) \quad (5)$$

where w is the basic channel bandwidth, ϖ_0 denotes the communication noise, $q_{X_{i,j}}$ and q_l are the transmission power of devices $X_{i,j}$ and D_l , respectively. $g_{X_{i,j},i}$ or $g_{l,i}$ is the channel gain from $X_{i,j}$ or D_l to the base station Z_i nearby. $\mathcal{D}_i \setminus X_{i,j}$ represents the device list involving all elements of $\mathcal{D}_i$ except $X_{i,j}$.

After the construction of the communication, the related devices $\mathcal{X}_i$ will upload data to the server. The data upload time for C_i in an edge server is calculated using

$$T_{\text{up},i}^{(\text{edge})} = \max_{l \in [1, n_i]} \frac{H_{i,l}^{(\text{up})}}{r_i(X_{i,l})} \quad (6)$$

where $H_{i,l}^{(\text{up})} \in \mathcal{H}_i^{(\text{up})}$ is the amount of data required by the l th device. The data download time from the cloud data center for task C_i is obtained by

$$T_{\text{down},i}^{(\text{edge})} = \frac{H_i^{(\text{down})}}{r_{\text{cloud}}} \quad (7)$$

where r_{cloud} represents the bandwidth between an edge server and the cloud.

Hence, the start time $T_{C_i}^{(\text{start})}$, the computation time T_{C_i} , and the end time $T_{C_i}^{(\text{end})}$ of C_i in Z_i can be calculated by (8), (9), and

$$T_{C_i}^{(\text{start})} = \max \left\{ \max_{k \in \Lambda_i^{(1)}} T_k^{(\text{end})}, \max_{k \in \Lambda_i^{(2)}} T_k^{(\text{start})}, T_{O_{i,1}}^{(\text{start})} \right\} \quad (8)$$

$$T_{C_i}^{(\text{end})} = \max \left\{ \max_{k \in \mathbf{\Lambda}_i^{(3)}} T_k^{(\text{end})}, T_{C_i}^{(\text{start})} + T_{C_i}, T_{J_i}^{(\text{end})} \right\}. \quad (10)$$

$$W_i^{(c)} = \sum_{l=1}^{n_i} \frac{q_{X_{i,l}} H_{i,l}^{(\text{up})}}{r_i(X_{i,l})} + P_{Z_i}^{(c)} \left(\alpha_1 \cdot \frac{R_{\text{cpu},i}}{V_{\text{cpu},Z_i}^{(\text{edge})}} + \alpha_2 \cdot \frac{R_{\text{mem},i}}{V_{\text{mem},Z_i}^{(\text{edge})}} \right) T_{C_i} \quad (11)$$

With the collaboration of the cloud server Z_i and the nearest edge server $E_k \in \mathcal{E}_i$, the data upload time for C_i is represented by

$$T_{\text{up},i}^{(\text{cloud})} = \max_{l \in [1, n_i]} \frac{H_{i,l}^{(\text{up})}}{r_k(X_{i,l})} + \sum_{l=1}^{n_i} \frac{H_{i,l}^{(\text{up})}}{r_{\text{cloud}}}. \quad (12)$$

The data download time between a cloud server and the cloud data center is negligible. The start time and the end time of C_i in a cloud server are shown by (8) and (10), respectively. The computation time of task C_i in a cloud server Z_i is defined by

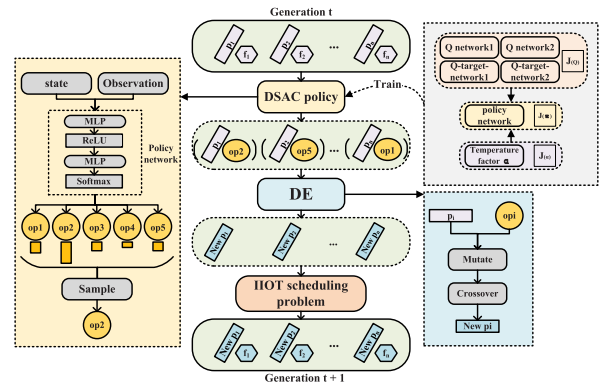
$$T_{C_i} = T_{\text{up},i}^{(\text{cloud})} + t_i. \quad (13)$$

Similar to (11), the energy consumption of finishing the computational task on a cloud server is defined by

$$W_i^{(c)} = \sum_{l=1}^{n_i} \frac{q_{X_{i,l}} H_{i,l}^{(\text{up})}}{r_k(X_{i,l})} + P_{Z_i}^{(c)} \left(\alpha_1 \cdot \frac{R_{\text{cpu},i}}{V_{\text{cpu},Z_i}^{(\text{cloud})}} + \alpha_2 \cdot \frac{R_{\text{mem},i}}{V_{\text{mem},Z_i}^{(\text{cloud})}} \right) T_{C_i} \quad (14)$$

C. Optimization Objective

In summary, the optimization objective in this article is to minimize the weighted sum of the overall makespan of computational tasks and manufacturing tasks and the overall energy



consumption, as represented by

$$f(\mathbf{X}, \mathbf{Y}, \mathbf{B}, \mathbf{Z}) = \lambda_1 \max \left\{ \max_{i \in [1, N]} T_{C_i}^{(\text{end})}, T^{(m)} \right\} + \lambda_2 \left(\sum_{i=1}^N W_i^{(c)} + W^{(m)} \right) \quad (15)$$

where λ_1, λ_2 are two weight factors within the scope of $(0, 1)$, and $\lambda_1 + \lambda_2 = 1$. $\mathbf{X}, \mathbf{Y}, \mathbf{B}, \mathbf{Z}$ denote the candidate sets of decision variables.

IV. PROPOSED DSAC-DE SCHEDULING ALGORITHM

This section provides the details of the proposed DSAC-DE algorithm. The main framework of the DSAC-DE is shown in Fig. 2. Suppose $\mathbf{p} = \{p_1, p_2, \dots, p_{N(p)}\}$ represents a group of individuals (i.e., solutions) for evolution, the DSAC policy selects a proper DE mutation operator for each individual, and the DE mutation operator generates offspring individuals.

A. DE Configuration

The real number encoding scheme is applied to uniformly represent the four decision variables groups. The coding length of an individual is $2N + 2\sum_{i=1}^N n_i$. Suppose the gene-bit of an individual is $\{\mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \mathbf{A}^{(3)}, \mathbf{A}^{(4)}\}$, each gene-bit $a_i^{(\kappa)} \in \mathbf{A}^{(\kappa)}$ is within the scope of $(0, 1)$, $\kappa \in [1, 4]$. The decoding rules between the gene-bits and the decision variables are explained as follows.

- 1) $a_i^{(1)} \in A^{(1)}, i \in [1, N]$. If $a_i^{(1)} > 0.5$, $B_i = 1$. Otherwise, $B_i = 0$.
- 2) $a_i^{(2)} \in A^{(2)}, i \in [1, N]$. If $B_i = 1$, $Z_i = \lfloor a_i^{(2)} \cdot |\mathbf{E}_i| \rfloor$. Otherwise, $Z_i = \lfloor a_i^{(2)} \cdot |\mathbf{S}| \rfloor$.
- 3) $a_i^{(3)} \in A^{(3)}, i \in [1, \sum_{i=1}^N n_i]$. $Y_{i,j} = a_{\sum_{k=1}^i n_k + j}^{(3)}$.
- 4) $a_i^{(4)} \in A^{(4)}, i \in [1, \sum_{i=1}^N n_i]$. $X_{i,j} = \lfloor a_{\sum_{i=1}^N n_k + j}^{(4)} \cdot |\mathbf{D}| \rfloor$.

DE generates the offspring individuals via differential mutation and differential crossover [28]. The five differential mutation operators adopted in this article are shown in

$$p_i^{(t+1)} = \begin{cases} 1) & p_{\text{gbest}}^{(t)} + F \cdot (p_{r_1}^{(t)} - p_{r_2}^{(t)}) \\ 2) & p_{r_1}^{(t)} + F \cdot (p_{r_2}^{(t)} - p_{r_3}^{(t)}) + F \cdot (p_{r_4}^{(t)} - p_{r_5}^{(t)}) \\ 3) & p_{\text{gbest}}^{(t)} + F \cdot (p_{r_1}^{(t)} - p_{r_2}^{(t)}) + F \cdot (p_{r_3}^{(t)} - p_{r_4}^{(t)}) \\ 4) & p_i^{(t)} + F \cdot (p_{\text{gbest}}^{(t)} - p_i^{(t)}) + F \cdot (p_{r_1}^{(t)} - p_{r_2}^{(t)}) \\ 5) & p_{r_1}^{(t)} + F \cdot (p_{r_2}^{(t)} - p_{r_3}^{(t)}) \end{cases} \quad (16)$$

where $p_i^{(t)}$ represents the offspring individual i in the t th iteration. $p_{\text{gbest}}^{(t)}$ is the global best individual ever find. $F \in [0, 2]$ is a scaling factor, and r_1, r_2, r_3, r_4, r_5 are five random variables within the scope of $[1, N^{(p)}]$. In each evolutionary iteration, the proposed DSAC policy selects different mutation operators $N^{(p)}$ times to generate $N^{(p)}$ offspring individuals, respectively.

The differential crossover following the DE mutation operator determines how much the parent individual $p_i^{(t)}$ is inherited by the offspring $p_i^{(t+1)}$, i.e., $p_i^{(t+1)} = p_i^{(t)}$, if $r_j \leq \text{CR}$, $\forall j \in [1, L]$, where L denotes the length of the codes. $r_j \in (0, 1)$ is a random variable, and $\text{CR} \in (0, 1)$ is a given crossover rate.

B. Dec-POMDP Modeling

The evolutionary process is modeled as a Dec-POMDP, which can be characterized by a tuple $\langle G, U, P, r, \Psi, \pi, \gamma \rangle$. Each individual is regarded as an agent, and the environment is the population and the DE configuration. $g \in G$ is the global state for all individuals. Each individual selects an action $u_i \in U$ to form a joint action $(u_1, u_2, \dots, u_{N^{(p)}})$. Each action u_i represents the serial number of a candidate differential mutation operator. With five differential mutation operators, the action space is 5. The state of the environment updates according to the state transition probability P and the joint action U . All individuals share the same reward function $r(g, u)$. $\pi_i \in \pi$ is the policy model for the individual p_i . p_i obtains an observation $\psi_i \in \Psi$ after the state transition. The relationship among g, u_i, ψ_i, π_i is represented by $u_i = \pi_i(g || \psi_i)$, where $||$ is the vector concatenation. $\gamma \in (0, 1)$ represents the discount factor.

1) State and Observation: The population state in the t th iteration is represented by $g^{(t)} = \langle g_{\text{mean}}^{(t)}, g_{\text{std}}^{(t)}, g_{\text{op1}}^{(t)}, g_{\text{op2}}^{(t)}, g_{\text{op3}}^{(t)}, g_{\text{op4}}^{(t)}, g_{\text{op5}}^{(t)} \rangle$, where $g_{\text{mean}}^{(t)}, g_{\text{std}}^{(t)}$ are the mean and the standard deviation of the population. They are normalized to $(0, 1)$ with the worst objective value in the population. $g_{\text{opi}}^{(t)}, i \in [1, 5]$ denotes the history utilization of the operator opi , and $g_{\text{opi}}^{(t)} = \langle NS_{\text{opi},1}^{(t)}, NS_{\text{opi},2}^{(t)}, NS_{\text{opi},3}^{(t)} \rangle$. $NS_{\text{opi},j}^{(t)}, j \in [1, 3]$ represents the time of successful evolution using opi , and $j \in [1, 3]$ means three perspectives to measure the success.

(a) If the offspring is better than the parent after applying opi in t th iteration, $NS_{\text{opi},1}^{(t+1)} = NS_{\text{opi},1}^{(t)} + 1$.

(b) If the offspring beats the global best one, $NS_{\text{opi},2}^{(t+1)} = NS_{\text{opi},2}^{(t)} + 1$.

(c) If the offspring is better than the average objective value, $NS_{\text{opi},3}^{(t+1)} = NS_{\text{opi},3}^{(t)} + 1$.

Hence, the total dimension of $g^{(t)}$ is 17 ($17 = 1+1+5*3$).

The observation identifies each individual and reflects the direction of evolution. The observation of p_i is represented by $\psi_i^{(t)} = \langle Df_{i,\text{gbest}}^{(t)}, Df_{i,r_1}^{(t)}, Df_{i,r_2}^{(t)}, Df_{i,r_3}^{(t)}, Df_{i,r_4}^{(t)}, Df_{i,r_5}^{(t)} \rangle$, where $Df_{i,\text{gbest}}^{(t)}$ is the absolute value of the difference between the objective value $f_i^{(t)}$ of $p_i^{(t)}$ and the global best objective value $f_{\text{gbest}}^{(t)}$. $Df_{i,r_j}^{(t)}, j \in [1, 5]$ represents the absolute value of the objective difference between individuals indexed by i and r_j , and $r_1, r_2, \dots, r_5$ are the five random parameters of the differential mutation operators. The total dimension of $\psi_i^{(t)}$ is 6.

2) Reward Function: The target of the DRL is to obtain higher rewards, and we formulate a reward function to optimize the objective. The reward function is represented by

$$r_t = \begin{cases} \frac{K_1}{f_{\text{gbest}}^{(t)}}, & t = \text{MAXGEN} \\ K_2 \cdot (f_{\text{gbest}}^{(t-1)} - f_{\text{gbest}}^{(t)}), & t \neq \text{MAXGEN} \end{cases} \quad (17)$$

where MAXGEN is the maximum number of evolutionary iterations, K_1, K_2 are two given parameters. $f_{\text{gbest}}^{(t)}$ is the global best objective in the t th iteration. The incremental rewards (r_t when $t \neq \text{MAXGEN}$) can accelerate the convergence speed for training the DRL model while the final reward (r_t when $t = \text{MAXGEN}$) indicates the best fitness value ever found. This article applies $K_2 = 4 \cdot K_1 / (f_{\text{gbest}}^{(0)})^2$ to restrict the impact of the final reward to be higher than that of the incremental reward, as the global best-searched objective determines the scheduling performance. It is proved as follows.

Proposition 1: A sufficient condition for the influence of the final reward to be greater than that of the incremental reward is that $K_2 \leq 4 \cdot K_1 / (f_{\text{gbest}}^{(0)})^2$.

Proof 1: Given that $K_2 \leq 4 \cdot K_1 / (f_{\text{gbest}}^{(0)})^2$. Based on Cauchy's inequality, then we have

$$\frac{K_2}{K_1} \leq \frac{4}{(f_{\text{gbest}}^{(0)})^2} \leq \frac{1}{(f_{\text{gbest}}^{(0)} - f_{\text{gbest}}^{(\text{MAXGEN})}) f_{\text{gbest}}^{(\text{MAXGEN})}} \quad (18)$$

$$\begin{aligned} \text{i.e., } \frac{K_1}{f_{\text{gbest}}^{(\text{MAXGEN})}} &\geq K_2 \left(f_{\text{gbest}}^{(0)} - f_{\text{gbest}}^{(\text{MAXGEN})} \right) \\ &= K_2 \sum_{t=0}^{\text{MAXGEN}} \left(f_{\text{gbest}}^{(t-1)} - f_{\text{gbest}}^{(t)} \right) \end{aligned} \quad (19)$$

which concludes the proof. Equation (19) means that the influence of the final reward is greater than or equal to that of the incremental rewards.

C. DSAC Policy Model

The DSAC policy is described by (20). It is learned by neural network models, which convert $g^{(t)}, \psi_i^{(t)}$ into the probability

distribution of five differential mutation operators (i.e., actions).

$$\begin{aligned} h_1 &= \text{ReLU} \left(w_1(g^{(t)} \parallel \psi_i^{(t)}) + b_1 \right) \\ h_2 &= w_2 h_1 + b_2 \end{aligned} \quad (20)$$

where $\text{ReLU}(x) = \max(0, x)$ is the ReLU activation function, w_1, w_2, b_1, b_2 are weight and bias parameters of two layers of multilayer perceptron (MLP), $\parallel$ means the concatenating operation. h_1, h_2 are two hidden variables. The dimension of h_1 is 128, and the dimension of h_2 is 5. Then, a SoftMax layer is introduced to scale each dimension of h_2 into the scope of $(0, 1)$, as demonstrated by

$$\rho_i = \frac{e^{h_{2,i}}}{\sum_{j=1}^5 e^{h_{2,j}}} \quad \forall i \in [1, 5] \quad (21)$$

where $\rho_i \in (0, 1)$ represents the probability of choosing the i th differential mutation operator shown in (16). The categorical distribution is constructed based on $\{\rho_1, \rho_2, \rho_3, \rho_4, \rho_5\}$, and the selected differential mutation operator is sampled from the distribution.

D. Training of the DSAC Policy

The training target is to learn the DSAC policy π to guide the selection of DE mutation operators. However, the randomness of the evolutionary search brings a large amount of noise to the training. Therefore, the maximum entropy learning that minimizes both the discounted reward and the maximum entropy is introduced to learn an optimal policy model π^* [29], as shown by

$$\pi^* = \arg\max_{\pi} \sum_t E_{(s_t, a_t) \sim \pi} [\gamma^t (r_t(s_t, a_t) + \alpha H(\pi(\cdot|s_t)))] \quad (22)$$

where $s_t = g_t \parallel \psi_t$ represents the concatenated state. π means the trajectory generated by the policy π . γ is the discounted factor. r_t represents the reward. $H(\pi(\cdot|s_t)) = -\log \pi(\cdot|s_t)$ is the information entropy of π at s_t , and α is a temperature factor that reflects the tradeoff between exploring more actions and obtaining higher rewards. A larger α means a greater optimization weight on action exploration.

To learn the optimal policy represented by (22), we apply the soft AC (SAC) framework in DRL, which totally contains five neural networks: a policy network $\pi(\cdot)$ defined in (20) and two pairs of critic networks $Q^{(1)}(\cdot)$, $Q^{(1, \text{target})}(\cdot)$ and $Q^{(2)}(\cdot)$, $Q^{(2, \text{target})}(\cdot)$. The critic networks evaluate the output of the policy model by estimating the Q -values, and we leverage two pairs of critic networks to avoid the overestimation of Q -values [30]. Hence, before training the policy model, it is necessary to train these critic networks first. The critic networks are updated based on the temporal difference (TD) method [31], and the loss function to update $Q^{(i)}(\cdot)$, $i \in \{1, 2\}$ is defined by

$$\begin{aligned} J(Q^{(i)}) &= E_{(s_t, a_t, s_{t+1}, r_t) \sim D} \left[\frac{1}{2} (Q^{(i)}(s_t, a_t) - (r_t + \gamma \min \right. \\ &\quad \times (Q^{(1, \text{target})}(s_{t+1}, a_{t+1}), Q^{(2, \text{target})}(s_{t+1}, a_{t+1})))^2 \left. \right] \end{aligned} \quad (23)$$

where D represents a replay buffer, (s_t, a_t, s_{t+1}, r_t) is a transition sampled from D , and $a_{t+1} = \arg\max_a \pi(a|s_{t+1})$ is the greedy action predicted by the policy model. The minimum value of $Q^{(1, \text{target})}(s_{t+1}, a_{t+1})$, $Q^{(2, \text{target})}(s_{t+1}, a_{t+1})$ is applied as the target Q -value to avoid overestimation.

Then, the policy model is updated to learn an optimal policy represented by (22). To adapt to the discrete action space in the cloud-edge-device collaborative task-scheduling problem, the loss function to train $\pi(\cdot)$ is designed by

$$\begin{aligned} J(\pi) &= E_{s_t \sim D} [\pi(s_t)^T [\alpha \log(\pi(s_t)) - \\ &\quad \min(Q^{(1)}(s_t, a_t), Q^{(2)}(s_t, a_t))]] \end{aligned} \quad (24)$$

where a_t is sampled based on $\pi(s_t)$. Equation (24) is the loss function representation of (22), and the detailed derivation can be found in [32]. The only difference between (24) and that introduced in [32] is that we directly use $\pi(s_t)^T$ as the action distribution considering the discrete action space. The temperature factor α is a hyperparameter, which can be updated using gradient descent. The loss function for α is defined by

$$\begin{aligned} J(\alpha) &= E_{a_t \sim \pi(s_t)} [-\alpha \log(\pi(s_t)) + H] \\ &= \pi(s_t)^T [-\alpha \log(\pi(s_t)) + H] \end{aligned} \quad (25)$$

where H is constant and represents the target entropy. Equation (25) means that $J(\alpha)$ is negatively correlated with $\pi(s_t)$. When the action selection tends to be deterministic (i.e., $\pi(s_t)$ is a large value), $J(\alpha)$ promotes the action exploration by increasing the value of α .

The training algorithm is summarized in Algorithm 1. η_θ, η_ϕ , and η_α are three learning rates. ∇ represents the gradient. $\tau \in (0, 1)$ is the soft updating rate. Line 1–Line 5 means the initialization. Line 7–Line 18 means the interaction between agents and the environment. Line 9–Line 14 generate offspring individuals and make the observation. Line 19–Line 24 represents the gradient step that updates neural networks and α with loss functions of (23), (24), and (25).

V. EXPERIMENTS AND DISCUSSIONS

A. Experimental Configuration

To validate the effectiveness of the proposed DSAC-DE in solving the cloud-edge-device task-scheduling problem, four artificial cases *case1*, *case2*, *case3*, and *case4* are introduced. The configurations of these four cases are reported in Table II. Take an automobile engine as a representative example [33], it contains nine manufacturing tasks. Considering the products with 10–40 discrete manufacturing steps, i.e., 10–40 tasks, the IIoT platform can take 10 orders as a batch to generate a scheduling solution. Therefore, there will be about 100 to 400 manufacturing tasks in a period. To demonstrate the device capacity restriction under different task scale, the number of manufacturing devices in the four cases are uniformly set as 300. As multiple computational tasks can be executed in parallel in a cloud/edge server with virtualization technology, the server capacity restriction is loose. To be simple, we set the number of cloud servers and distributed edge servers as the same as

Algorithm 1: Training Algorithm.

```

1 Initialize  $\theta_1, \theta_1^-$  in  $Q^{(1)}(\cdot), Q^{(1,target)}(\cdot)$ ;
2 Initialize  $\theta_2, \theta_2^-$  in  $Q^{(2)}(\cdot), Q^{(2,target)}(\cdot)$ ;
3 Initialize  $\phi$  in  $\pi(\cdot)$ ;
4 Initialize  $g_t$ ;
5  $D \leftarrow \emptyset$ ;
6 foreach  $t \in [1, MAXGEN]$  do
7   foreach interaction step do
8      $A_t \leftarrow \emptyset$ ;
9     foreach individual  $i$  in the population do
10      Obtain the individual observation  $\psi_t^{(i)}$ ;
11       $a_t = \pi_\phi(g_t || \psi_t^{(i)})$ ;
12      Append  $a_t$  to  $A_t$ ;
13       $D \leftarrow D \cup \{(g_t, \psi_t^{(i)}, a_t)\}$ ;
14    end
15    Receive  $g_{t+1}, r_t$  after outputting  $A_t$  to the environment;
16     $D \leftarrow D \cup \{(r_t, g_{t+1})\}$ ;
17     $g_t \leftarrow g_{t+1}$ ;
18  end
19  foreach gradient step do
20     $\theta_i \leftarrow \theta_i - \eta_\theta \nabla J(Q_i)$  for  $i \in \{1, 2\}$ ;
21     $\phi \leftarrow \phi - \eta_\phi \nabla J(\pi)$ ;
22     $\alpha \leftarrow \alpha - \eta_\alpha \nabla J(\alpha)$ ;
23     $\theta_i^- \leftarrow \tau \theta_i + (1 - \tau) \theta_i^-$  for  $i \in \{1, 2\}$ ;
24  end
25 end

```

TABLE II
CONFIGURATIONS OF FOUR CASES

No.	Case1	Case2	Case3	Case4
N	100	200	300	400
$ D $	300	300	300	300
$ C $	100	200	300	400
$ E $	100	200	300	400
Number of Type I constraint	98	152	237	340
Number of Type II constraint	255	496	727	967
Number of Type III constraint	339	663	999	1335
n_i	5	5	5	5
$T_{O_{i,j}}^{(proc)}$	[20, 200]s	[20, 200]s	[20, 200]s	[20, 200]s
t_i	[50, 200]s	[50, 200]s	[50, 200]s	[50, 200]s
H_i^{up}	[50, 500]MB	[50, 500]MB	[50, 500]MB	[50, 500]MB
H_i^{down}	[50, 500]MB	[50, 500]MB	[50, 500]MB	[50, 500]MB
$P_t^{(D)}$	2.1kW	2.1kW	2.1kW	2.1kW
$R_{cpu,i}$	[4, 16]cores	[4, 16]cores	[4, 16]cores	[4, 16]cores
$R_{mem,i}$	[4, 32]GB	[4, 32]GB	[4, 32]GB	[4, 32]GB
$V_{cpu,k}^{(cloud/edge)}$	[32,128]cores	[32,128]cores	[32,128]cores	[32,128]cores
$V_{mem,k}^{(cloud/edge)}$	[128, 256]GB	[128, 256]GB	[128, 256]GB	[128, 256]GB
$T_{X_{i,j-1}, X_{i,j}}^{(trans)}$	[18, 294]s	[18, 294]s	[18, 294]s	[18, 294]s
$q_{X_{i,j}}$	[27, 30]dBm	[27, 30]dBm	[27, 30]dBm	[27, 30]dBm
$g_{X_{i,j}}$	[0.8, 2.7]dB	[0.8, 2.7]dB	[0.8, 2.7]dB	[0.8, 2.7]dB
r_{cloud}	100Mb/s	100Mb/s	100Mb/s	100Mb/s
ω	500Mb/s	500Mb/s	500Mb/s	500Mb/s

the number of computational tasks in each artificial case. If the number of tasks and resources increases, they can be divided into multiple groups. The proposed algorithm can be applied to different groups simultaneously with only one-time training. The expected computing time and energy of manufacturing tasks and computational tasks are generated from two datasets, i.e., ClusterData 2011 traces [34] and Alibaba Cluster Data V2018 [35]. The server configurations of both cloud and edge are all from the SPECpower sjj2008 dataset.

The DSAC policy model is trained on *case1* and applied to the four cases to show its extendibility. MAXGEN for training

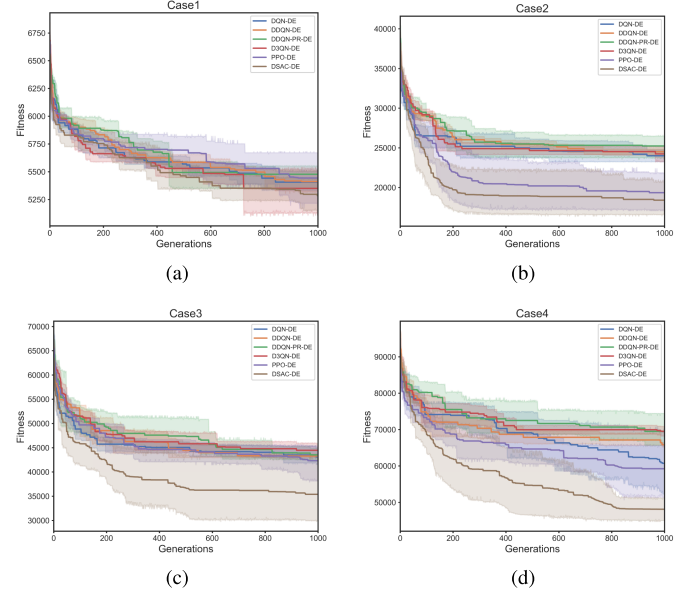


Fig. 3. Comparison results on searching curves. (a) Case1. (b) Case2. (c) Case3. (d) Case4.

TABLE III
PERCENTAGE OF THE IMPROVEMENTS ON THE AVERAGE OBJECTIVE VALUES OBTAINED BY THE DSAC-DE COMPARED WITH OTHER DRL-DES

	DQN-DE	DDQN-DE	DDQN-PR-DE	D3QN-DE	PPO-DE
Case 1	2.25%	2.33%	3.50%	1.21%	2.91%
Case 2	23.33%	24.91%	27.10%	24.10%	4.89%
Case 3	17.93%	18.25%	18.71%	20.42%	16.40%
Case 4	20.78%	26.94%	30.80%	30.82%	18.81%

The boldface values indicate the greatest improvements in the four cases.

is 200 in an episode, and MAXGEN for evaluation is 1000. The population size is 30. The experiments and algorithm comparisons are conducted using Python 3.7.1 and PyTorch 1.6.0 with CUDA 11.0. The running environment is Intel XEON Gold 6240 @2.4 GHz CPU, 128 GB RAM, and NVIDIA RTX 3080 GPU.

B. Comparison of Different DRL Algorithms on the Automatic Configuration of DE

In this section, the proposed DSAC policy model is compared to DQN [21], DDQN [24], DDQN with prioritized experience replay (DDQN-PR), Dueling DDQN (D3QN), and proximal policy optimization (PPO) to show its effectiveness on finding suitable differential mutation operators for solving the cloud–edge–device collaborative task-scheduling problem. All algorithms are based on the same architecture described in Fig. 2. The hidden units of the six DRL-DE algorithms are [32, 32], the learning rate is 0.0001, the buffer size is 40000, the batch size is 512, γ is set as 0.99, and α and β are 0.5 and 0.5, respectively. The models after 700 training episodes are applied.

Fig. 3 indicates the searching curves of DRL-DE algorithms, the shadow means the fluctuation range, and the line denotes the average values. It can be found that DSAC-DE converges faster than others, especially in *case2*, and DSAC-DE can find better solutions in all four cases. Table III reports the percentage of the improvements on the average objective values obtained by

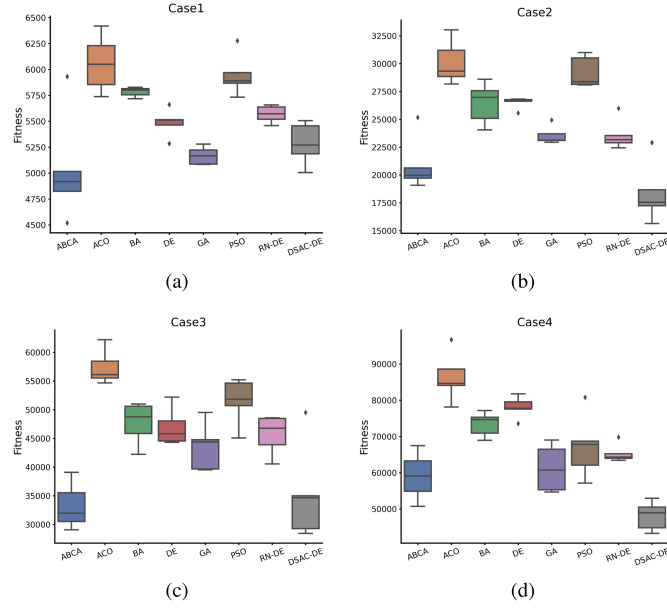


Fig. 4. Comparison results on final objective value. (a) Case1. (b) Case2. (c) Case3. (d) Case4.

TABLE IV
PERCENTAGE OF THE IMPROVEMENTS ON THE AVERAGE OBJECTIVE VALUES OBTAINED BY THE DSAC-DE COMPARED WITH OTHER EAs

	ABCA	ACO	BA	DE	GA	PSO	RNDE
Case 1	-4.82%	12.76%	8.62%	3.69%	-2.25%	11.13%	5.10%
Case 2	12.00%	38.90%	30.43%	30.56%	21.89%	37.02%	22.04%
Case 3	-5.99%	38.35%	25.80%	24.71%	18.81%	31.28%	22.51%
Case 4	18.59%	44.35%	34.47%	38.37%	21.45%	28.54%	26.39%

The boldface values indicate the greatest improvements in the four cases.

the DSAC-DE compared with the other five DRL-DE methods. The value shows it can reduce the average objective values by 30.82% at most compared to D3QN-DE in *case4*.

C. Comparison of the Proposed DSAC-DE to Typical EAs

The proposed DSAC-DE is compared to six typical EAs, including an artificial bee colony algorithm (ABCA), ACO, bees algorithm (BA), DE, GA, and PSO and a random configured DE (RN-DE). In RN-DE, the differential mutation operator is randomly selected for each individual in each evolutionary iteration.

The average results of these algorithms in 10 runs are shown in Fig. 4. RN-DE performs better than DE in *case2* and *case4*. This illustrates the advantage of applying multiple differential mutation operators. DSAC-DE shows far better than RN-DE. It proved that the DSAC policy learned from *case1* and can be well extended to other three cases. In addition, DSAC-DE performs remarkably better than ACO, BA, DE, GA, and PSO, and performs similar or even better than ABCA, which adopts additional local search iterations. Table IV reports the percentage of the improvements on the average objective values obtained by the DSAC-DE compared with other EAs, which indicates that

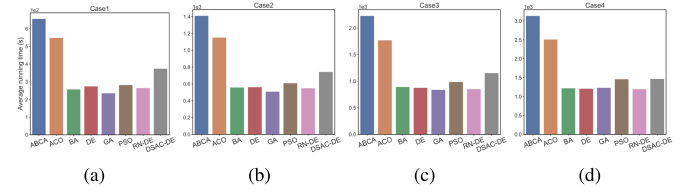


Fig. 5. Comparison results on average running time. (a) Case1. (b) Case2. (c) Case3. (d) Case4.

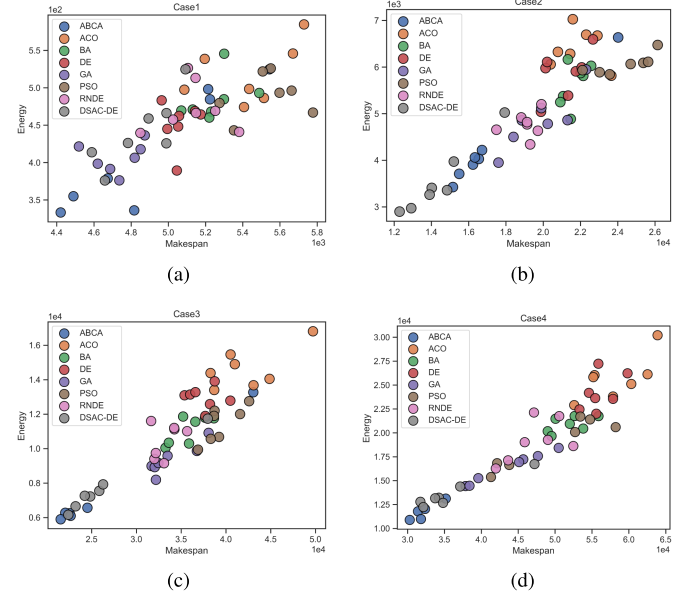


Fig. 6. Comparison results on the makespan and the total energy. (a) Case1. (b) Case2. (c) Case3. (d) Case4.

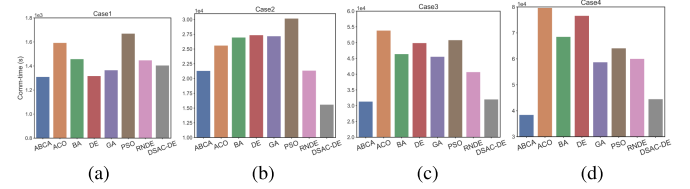


Fig. 7. Comparison results on the total communication time of CT. (a) Case1. (b) Case2. (c) Case3. (d) Case4.

the proposed DSAC-DE can reduce the average objective values by 44.35% at most compared to ACO in *case2*.

As shown in Fig. 5, the decision time of DSAC-DE is slightly higher than that of BA, DE, GA, and PSO and is much lower than that of ACO and ABCA with local search iterations.

The best seven solutions obtained by these algorithms are demonstrated in Fig. 6. The results of DASC-DE mainly gather in the lower-left side of Fig. 6, which means that DSAC-DE can always find solutions with the lowest makespan and energy consumption. In addition, the total communication time of hybrid tasks is indicated in Fig. 7. It can be found that ABCA and DSAC-DE can find solutions with lower communication time

TABLE V
RESOURCE AVAILABILITIES OF EIGHT CASES

No.	Case1	Case1A	Case2	Case2A	Case3	Case3A	Case4	Case4A
$N^{(m)}$	100	100	200	200	300	300	400	400
$ C $	100	95	200	195	300	300	400	395
$ E $	100	100	200	195	300	295	400	395

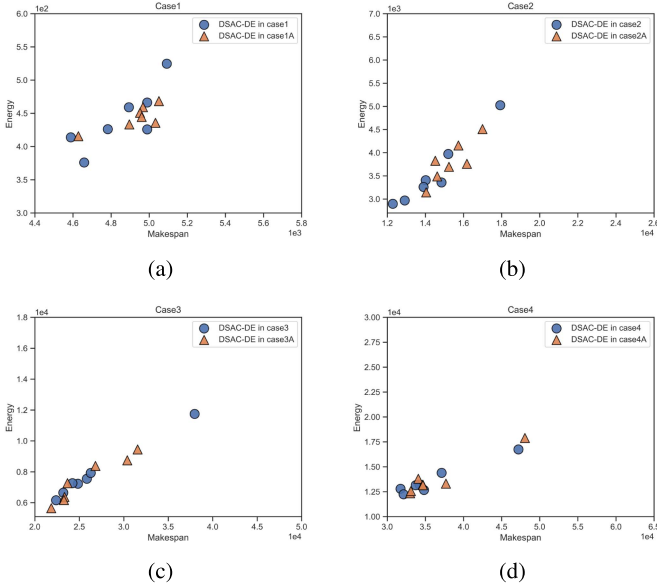


Fig. 8. Comparison results on cases with diverse resource availabilities. (a) Case1. (b) Case2. (c) Case3. (d) Case4.

than others, which also shows that their scheduling solutions are more reasonable in practice.

D. Adaptability to Changes in Resources

To explore whether the performance of a trained DSAC policy will decline when the resource availability changes, the four cases are modified to *case1A*–*case4A* by randomly reducing the available servers, as reported in Table V. The experiments of the DSAC policy without re-training on all of the eight cases are demonstrated in Fig. 8. It can be found that there is no apparent difference between the two types of points in four subfigures of Fig. 8. The solution quality is well maintained with changing resource availabilities. Therefore, the results validate that the DSAC policy is able to adapt to changes in the availability of resources.

To conclude, the proposed DSAC-DE performs better and is more stable than DRL-based EAs and typical EAs. Without tedious parameter tuning and operator selection, it is able to find suitable scheduling solutions for different scenarios with only one-time training.

VI. CONCLUSION

This article studied the cloud–edge–device collaborative task-scheduling problem with three kinds of task constraints. To solve this problem in a parameter-free way, a DSAC-DE algorithm was proposed. The evolutionary process was modeled as a Dec-POMDP. The maximum entropy learning in DSAC reduced

the influence of searching randomness of DE and alleviated overfitting. Experimental results verified the effectiveness of the proposed algorithm in different scales of cloud–edge–device collaborative task-scheduling problems and proved its superiority compared to other typical EAs and DRL-based EAs.

Future works include developing an end-to-end perception of the scheduling scene to generate an adaptable state and reward and designing parallel strategies to accelerate both the training and searching process.

REFERENCES

- [1] L. Da Xu, W. He, and S. Li, “Internet of Things in industries: A survey,” *IEEE Trans. Ind. Informat.*, vol. 10, no. 4, pp. 2233–2243, Nov. 2014.
- [2] E. Sisinni, A. Saifullah, S. Han, U. Jennehag, and M. Gidlund, “Industrial Internet of Things: Challenges, opportunities, and directions,” *IEEE Trans. Ind. Informat.*, vol. 14, no. 11, pp. 4724–4734, Nov. 2018.
- [3] A. Martins, J. Lucas, H. Costelha, and N. Carlos, “Developing an OPC UA server for CNC machines,” *Procedia Comput. Sci.*, vol. 180, pp. 561–570, 2021.
- [4] Y. Lu, X. Huang, K. Zhang, S. Maharjan, and Y. Zhang, “Communication-efficient federated learning for digital twin edge networks in Industrial IOT,” *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5709–5718, Aug. 2021.
- [5] N. Srinidhi, S. D. Kumar, and K. Venugopal, “Network optimizations in the Internet of Things: A review,” *Eng. Sci. Technol.*, vol. 22, no. 1, pp. 1–21, 2021.
- [6] D. A. Chekired, L. Khoukhi, and H. T. Mouftah, “Industrial IOT data scheduling based on hierarchical fog computing: A key for enabling smart factory,” *IEEE Trans. Ind. Informat.*, vol. 14, no. 10, pp. 4590–4602, Oct. 2018.
- [7] W. Mao, Z. Zhao, Z. Chang, G. Min, and W. Gao, “Energy-efficient industrial Internet of Things: Overview and open issues,” *IEEE Trans. Ind. Informat.*, vol. 17, no. 11, pp. 7225–7237, Nov. 2021.
- [8] L. Chen, P. Zhou, L. Gao, and J. Xu, “Adaptive fog configuration for the industrial Internet of Things,” *IEEE Trans. Ind. Informat.*, vol. 14, no. 10, pp. 4656–4664, Oct. 2018.
- [9] H. Yuan, J. Bi, and M. Zhou, “Multiqueue scheduling of heterogeneous tasks with bounded response time in hybrid green IaaS clouds,” *IEEE Trans. Ind. Informat.*, vol. 15, no. 10, pp. 5404–5412, Oct. 2019.
- [10] Y. Xie et al., “A novel directional and non-local-convergent particle swarm optimization based workflow scheduling in cloud–edge environment,” *Future Gener. Comput. Syst.*, vol. 97, pp. 361–378, 2019.
- [11] D. Liu, X. Chen, Z. Zhou, and Q. Ling, “HierTrain: Fast hierarchical edge AI learning with hybrid parallelism in mobile-edge-cloud computing,” *IEEE Open J. Commun. Soc.*, vol. 1, pp. 634–645, May 2020. doi: 10.1109/OJCOMS.2020.2994737.
- [12] S. Luo, “Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning,” *Appl. Soft Comput.*, vol. 91, 2020, Art. no. 106208.
- [13] G. Zhang, Y. Hu, J. Sun, and W. Zhang, “An improved genetic algorithm for the flexible job shop scheduling problem with multiple time constraints,” *Swarm Evol. Comput.*, vol. 54, 2020, Art. no. 100664.
- [14] M. M. Ahmadian, A. Salehipour, and T. Cheng, “A meta-heuristic to solve the just-in-time job-shop scheduling problem,” *Eur. J. Oper. Res.*, vol. 288, no. 1, pp. 14–29, 2021.
- [15] Y. Cao, S. Wang, L. Kang, and Y. Gao, “A TQCS-based service selection and scheduling strategy in cloud manufacturing,” *Int. J. Adv. Manuf. Technol.*, vol. 82, no. 1, pp. 235–251, 2016.
- [16] W. Li et al., “Subtask scheduling for distributed robots in cloud manufacturing,” *IEEE Syst. J.*, vol. 11, no. 2, pp. 941–950, Jun. 2017.
- [17] H. Akbaripour, M. Houshmand, T. Van Woensel, and N. Mutlu, “Cloud manufacturing service selection optimization and scheduling with transportation considerations: Mixed-integer programming models,” *Int. J. Adv. Manuf. Technol.*, vol. 95, no. 1, pp. 43–70, 2018.
- [18] X. Wang, L. Zhang, T. Lin, C. Zhao, K. Wang, and Z. Chen, “Solving job scheduling problems in a resource preemption environment with multi-agent reinforcement learning,” *Robot. Comput.-Integr. Manuf.*, vol. 77, 2022, Art. no. 102324.
- [19] Y. Jin, H. Wang, T. Chugh, D. Guo, and K. Miettinen, “Data-driven evolutionary optimization: An overview and case studies,” *IEEE Trans. Evol. Comput.*, vol. 23, no. 3, pp. 442–458, Jun. 2019.

- [20] R. Chen, B. Yang, S. Li, and S. Wang, "A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem," *Comput. Ind. Eng.*, vol. 149, 2020, Art. no. 106778.
- [21] Z. Tan and K. Li, "Differential evolution with mixed mutation strategy based on deep reinforcement learning," *Appl. Soft Comput.*, vol. 111, 2021, Art. no. 107678.
- [22] T. Visutarnom, T.-C. Chiang, A. Konak, and S. Kulturel-Konak, "Reinforcement learning-based differential evolution for solving economic dispatch problems," in *Proc. IEEE Int. Conf. Ind. Eng. Eng. Manage.*, 2020, pp. 913–917.
- [23] T. Zhang, H. Wang, B. Yuan, Y. Jin, and X. Yao, "Surrogate-assisted evolutionary q-learning for black-box dynamic time-linkage optimization problems," *IEEE Trans. Evol. Comput.*, to be published, doi: [10.1109/TEVC.2022.3179256](https://doi.org/10.1109/TEVC.2022.3179256).
- [24] M. Sharma, A. Komninos, M. López-Ibáñez, and D. Kazakov, "Deep reinforcement learning based parameter control in differential evolution," in *Proc. Genet. Evol. Comput. Conf.*, 2019, pp. 709–717.
- [25] Z. Xu, G. Han, L. Liu, M. Martínez-García, and Z. Wang, "Multi-energy scheduling of an industrial integrated energy system by reinforcement learning-based differential evolution," *IEEE Trans. Green Commun. Netw.*, vol. 5, no. 3, pp. 1077–1090, Sep. 2021.
- [26] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.
- [27] L. Ren, Y. Laili, X. Li, and X. Wang, "Coding-based large-scale task assignment for industrial edge intelligence," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2286–2297, Oct.–Dec. 2020.
- [28] K. V. Price, "Differential evolution," in *Handbook of Optimization*. Berlin, Germany: Springer, 2013, pp. 187–214.
- [29] T. Haarnoja et al., "Soft actor-critic algorithms and applications," 2018, *arXiv:1812.05905*.
- [30] G. An, S. Moon, J.-H. Kim, and H. O. Song, "Uncertainty-based offline reinforcement learning with diversified q-ensemble," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 7436–7447, 2021.
- [31] S. Blakeman and D. Mareschal, "A complementary learning systems approach to temporal difference learning," *Neural Netw.*, vol. 122, pp. 218–230, 2020.
- [32] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
- [33] Y. Laili, C. Peng, Z. Chen, F. Ye, and L. Zhang, "Concurrent local search for process planning and scheduling in the industrial internet-of-things environment," *J. Ind. Inf. Integration*, vol. 28, 2022, Art. no. 100364.
- [34] Y. Lin, A. Barker, and S. Ceesay, "Exploring characteristics of inter-cluster machines and cloud applications on Google clusters," in *Proc. IEEE Int. Conf. Big Data*, 2020, pp. 2785–2794.
- [35] J. Guo et al., "Who limits the resource efficiency of my datacenter: An analysis of Alibaba datacenter traces," in *Proc. Int. Symp. Qual. Service*, 2019, pp. 1–10.



Yuanjun Laili (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in automation control from the School of Automation Science and Electrical Engineering, Beihang University, Beijing, China, in 2009, 2012, and 2015, respectively.

She is currently an Associate Professor with the School of Automation Science and Electrical Engineering, Beihang University, and with Zhongguancun Laboratory, Beijing. She is also a Chief Scientist of the National Key R&D Program of China. Her main research interests include the areas of intelligent optimization, modeling, and simulation of manufacturing systems.

Dr. Laili is a member of The Society For Modeling and Simulation International (SCS). She has won the "Young Talent Lift Project" supported by the China Association for Science and Technology and the "Young Simulation Scientist Award" from SCS. She is an Associate Editor for the *International Journal of Modeling, Simulation, and Scientific Computing* and *Cogent Engineering*.



Xiaohan Wang received the B.S. degree in 2019 from Beihang University, Beijing, China, where he is currently working toward the Ph.D. degree with the School of Automation Science and Electrical Engineering.

His research interests include scheduling, deep reinforcement learning, and machine learning-based optimization.

Mr. Wang received the recommended test-free qualification for Ph.D. in 2018 and obtained the outstanding graduate of Beihang University in 2019.



Lin Zhang (Senior Member, IEEE) received the B.S. degree from the Department of Computer and System Science, Nankai University, Tianjin, China, in 1986, and the M.S. and Ph.D. degrees from the Department of Automation, Tsinghua University, Beijing, China, in 1989 and 1992, respectively.

He is currently a Professor with Beihang University, Beijing. From 2002 to 2005, he was with the US Naval Postgraduate School as a Senior Research Associate of the US National Research Council. He authored or coauthored more than 300 papers and 20 books and chapters. His research interests include service-oriented modeling and simulation, cloud manufacturing and simulation, model engineering, model-based systems engineering, cyber-physical systems, M&S for manufacturing systems, etc.

Dr. Zhang was the President of the Society for Modeling and Simulation International (SCS), the Executive Vice-President of the China Simulation Federation (CSF), and the President of the Federation of Asian Simulation Societies (ASIASIM). He is a Fellow of SCS, ASIASIM and CSF, a Chief Scientist of the National 863 Program and National Key R&D Program of China. He is the Director of Engineering Research Center of Complex Product Advanced Manufacturing Systems, Ministry of Education of China, and Editor-in-Chief and Associate Editor of eight peer-reviewed international journals. He is a recipient of the National Award for Excellent Science and Technology Books, the Outstanding Individual Award of National High-Tech R&D Program, the National Excellent Scientific and Technological Workers Awards, and the SCS Outstanding Professional Contribution Award.



Lei Ren (Member, IEEE) received the Ph.D. degree in computer science from the Institute of Software, Chinese Academy of Sciences, Beijing, China, in 2009.

He is currently a Professor with the School of Automation Science and Electrical Engineering, Beihang University, Beijing, and also with Zhongguancun Laboratory, Beijing. His research interests include neural networks and deep learning, time-series analysis, and industrial artificial intelligence applications.

Dr. Ren is an Associate Editor for the *IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS* and other international journals.